

라서라xu ㄱ 감 ag 색의차별사분점의상위선생물상접역용색응산생산사생생의영일비시
백요응저생비자일생비응하일생생사노이응아비응선

LW7u가배간구결
Lk관트7qg[W] 계표기5근결 >5계고A5공7B?>물AA5=9=9
Lgnc

g고-결계간계고관간c-고계개gnc 2		Vi i		kh 6gnc 1관갈h-구가계개g고-결계간계고관간	
c-고계개7gnc				5	
7					
kh 6gnc	gnc	1	5	2	5
7				1공관가경-구가계개	
L] 까다구관		ci nr		5gnc	
7				7	

MH8A t 감

L

7

L

c gnc L

공: R 공 = 1 2

가가개다관 Lk관가개P 거개&매공:

가가개R ≥관 Lk관가개P 거개&다공 =

≥R: 7

L

유사 포인트 (거리 ≤ r)

↓ 높은 확률로

[같은 해시 버킷]

↑ 낮은 확률로

다른 포인트 (거리 > cr)

MH8 t 감

L

layer 1: $h_1 = [h^{(1)}_1, h^{(1)}_2, \dots, h^{(1)}_k] \rightarrow$ 해시값 1
(k개 함수의 연결)

layer 2: $h_2 = [h^{(2)}_1, h^{(2)}_2, \dots, h^{(2)}_k] \rightarrow$ 해시값 2

...

layer L: $h_L = [h^{(L)}_1, h^{(L)}_2, \dots, h^{(L)}_k] \rightarrow$ 해시값 L

L

1. 쿼리 q의 각 레이어별 해시값 계산

2. 각 레이어에서 같은 해시 버킷의 모든 점 찾기
3. 찾은 모든 점들 중 가장 가까운 K개 선택

시간복잡도: $O(L \times \text{hash_computation} + \text{후보_수})$

MH8xu ㄱ ㄱ

L

gnc L

: 7 L

=7 Lg

>7 L

L

전통 LSH:

- 레이어 1: 해시 (010110...)
- 레이어 2: 해시 (101010...)
- 레이어 3: 해시 (111001...)
- 각 레이어에서 따로 검색

PM-LSH:

- 모든 레이어의 해시를 연결한 하나의 키: 010110|101010|111001
- 검색 시:
- 1단계: 해시 010110* 으로 시작하는 모든 항목 (레이어 1)
- 2단계: 010110|101010* 으로 시작하는 모든 항목 (레이어 2)
- 3단계: 010110|101010|111001* 정확히 일치

MH8Mxu ㄱ ㄱ

L

```
procedure BUILD_PM_LSH_INDEX(vectors, k, L):
    // k: 각 레이어의 함수 개수
    // L: 레이어 개수

    // 1단계: LSH 함수족 생성
    for layer = 1 to L:
        for func_idx = 1 to k:
            h[layer][func_idx] = create_random_projection()

    // 2단계: 접두사 트리 구성
    prefix_tree = TrieNode()

    for vector v in vectors:
        key = ""
        for layer = 1 to L:
            // 해시값 계산
            hash_value = compute_hash(v, h[layer], k)
            key = key + "|" + hash_value // 접두사 연결

        // 트라이에 삽입
        insert_to_trie(prefix_tree, key, v)

    return prefix_tree
```

L

```
procedure SEARCH_PM_LSH(query_q, K, early_termination_threshold):
    candidates = []
    prefix = ""

    for layer = 1 to L:
        // 현재 레이어의 해시값 계산
        hash_value = compute_hash(q, h[layer], K)
        prefix = prefix + "|" + hash_value
```

```
// 점두사와 일치하는 모든 벡터 찾기
matching_vectors = search_prefix_in_trie(
    prefix_tree, prefix
)

// 거리 계산 및 후보 수집
for v in matching_vectors:
    dist = distance(q, v)
    candidates.append((v, dist))

// 조기 종료 (충분한 좋은 결과)
if len(candidates) >= early_termination_threshold:
    break

// 가장 가까운 k개 반환
return top_K_by_distance(candidates, K)
```

MH8N

m계고정 k관변수 관계1 2

gnc

```
h_a,b(v) = floor((a · v + b) / r)
```

- 여기서:
- a: 무작위 벡터 (각 차원: $N(0,1)$)
 - b: $[0, r)$ 범위의 무작위 스칼라
 - r: 버킷 크기 (파라미터)
 - v: 입력 벡터

거리 메트릭: L2 (유클리드)

gnc L

```
h_a(v) = sign(a · v)
```

- 여기서:
- a: 무작위 벡터
 - sign(): 양수면 1, 음수면 0

확률: $P(h_a(x) = h_a(y)) = 1 - \theta(x,y)/\pi$
(θ 는 벡터 간 각도)

MH8P

계 g L

- 더 많은 함수 (큰 k):
- 더 정확한 구분 (낮은 거짓양성)
 - 높은 계산 비용
 - 낮은 거짓음성 (실제 유사점 놓치기)

- 더 많은 레이어 (큰 L):
- 더 높은 회수율(recall)
 - 더 큰 메모리
 - 더 많은 계산

최적화:

k + L = 상수 (메모리 제약)

k를 높여 정확도 향상

L을 높여 회수율 향상

관 L

- 작은 r (더 많은 버킷):
- 낮은 거짓양성
 - 높은 거짓음성

- 큰 r (작은 버킷):
- 높은 거짓양성
 - 낮은 거짓음성

권장: 데이터의 타입 거리 분포에 맞춰 선택

MH8

공간 L

루트
└ 0
| └ 1
| | └ 0 → [벡터 ID들]
| | └ 1 → [벡터 ID들]
| └ 0
| └ 1 → [벡터 ID들]
└ 1
└ ...

L

전통 LSH (L개 해시 테이블):
메모리 = L × (평균 버킷 크기 × 포인터)

PM-LSH (단일 트리):
메모리 = 트리 노드 수 × (자식 수 + 데이터)

일반적으로 PM-LSH가 10~30% 더 효율적

MH8V

L

- 인덱싱: $O(L \times k \times n \times d)$
- n: 벡터 개수
 - d: 벡터 차원
 - k: 함수 개수
 - L: 레이어 개수

검색: $O(L \times k + \text{평균_후보_수} \times d)$

- 조기 종료로 평균 L을 줄일 수 있음

L

벡터 저장: $O(n \times d)$
LSH 구조: $O(L \times k \times d)$ (함수 저장)
트리: 0(계약 가능)

MH8 xu 7 감 잡 ov 감

	kh 6gnc	ci nr
4666646666646666-H		
	는	

MH8A;

gnc L

procedure SEARCH_FILTERED_PM_LSH(
query_q, K, filter_predicate

```

):
    candidates = []

    for layer = 1 to L:
        hash_value = compute_hash(q, h[layer])
        prefix = current_prefix + hash_value

        // 점두사와 일치하는 벡터
        matching_vectors = search_prefix(prefix_tree, prefix)

        for v in matching_vectors:
            // 필터 조건 확인
            if not filter_predicate(v):
                continue

            // 거리 계산
            dist = distance(q, v)
            candidates.append((v, dist))

        // 충분한 결과를 얻으면 종료
        if len(candidates) >= K:
            break

    return top_K(candidates, K)

```

L

필터 조건 "category == 'electronics'":
- 전체 벡터의 30% 선택

LSH 해시로 찾은 후보: 100개
- 그 중 필터 만족: 30개 (예상)

정확도는 유지되지만 후보 집합이 감소
→ 거짓음성 증가 가능성

MH8AA

L

처음 N개 레이어만 검색:
- 빠른 응답
- 낮은 정확도

필요시 추가 레이어로 확장:
- 더 많은 후보 수집
- 회수율 향상
- 응답 시간 증가

MH8AH 2Au 6A; ; ; 3

구성 시간:
HNSW: 약 30분
PM-LSH: 약 5분

메모리:
HNSW: 약 4GB
PM-LSH: 약 3GB

검색 속도:
HNSW: 0.5ms/쿼리
PM-LSH: 1~2ms/쿼리

정확도 (top-10):
HNSW: 95%
PM-LSH: 90%

: 7	gnc	L	5gnc	S	
	S				
=7		L		S	5(≥-구개교개)
	gnc S				
>7		L			S
?7	gnc	L	5	gnc	S
A7	Lgnc		5		S
B7		L		5gnc	
	S				
D7	L		5	gnc	S
E7		Lkh 6gnc	ci nr		S
H7	Lci nr		kh 6gnc	S	